University of the West Indies, Cave Hill Campus
Department of Computer Science, Mathematics and Physics

LAB 2 - Programmable Logic Devices (PLDs)

Name: Janelle Mentor

Date of Submission: 28th February 2026

Course Code: ELET2235 Automation Technology & Applications

TABLE OF CONTENTS

INTRODUCTION

A Programmable logic device (PLD) is an electronic component or integrated circuit that contains an array of logic elements and interconnections that can be configured/programmed by users to perform specific logic functions. PLDs can be programmed and reprogrammed to implement different logic circuits unlike fixed logic devices such as logic gates or flip-flops. They are programmed using hardware description languages such as Verilog or VHDL or graphical user interface software tools. The logic elements are normally simple combinational or sequential circuits, such as AND, OR, NOT, and XOR gates and the interconnections are usually programmable switches or multiplexers that can connect the inputs and outputs of the logical elements in different ways. There are many types of PLDs that vary from simple Programmable Read-Only Memory (PROM), Programmable Array Logic (PALs) and Programmable Logic Arrays (PLAs) to complex CPLDs (Complex Programmable Logic Arrays) and FPGAs (Field Programmable Gate Arrays), which offers different levels of complexity and reusability.

PLDs are integral in creating efficient, flexible digital systems for microprocessors, embedded systems, enabling rapid prototyping and dynamic reconfigurability. They offer performance and cost effectiveness, allowing parallel processing and easy updates to functionality, which are beneficial over fixed logic gates.

This lab will focus on FPGA, which is the most complex and largest PLD that contains millions of logic elements and interconnections. They are usually programmed using SRAM (Static Random-Access Memory) technology, which means that they need an external memory device to store their configuration data. FPGAs consist of a number of sub-blocks of components called Combinational Logic Blocks (CLBs), which are the primary component of FPGAs. They are made up of 4-input LUTs (Look-up Tables), which can be split into 2 or 3-input LUTS, full-adder, and D flip-flop.

The objective of this lab was to design and implement a simple Configurable Logic Block capable of performing a simple logic function. The function to be implemented is:

$$F = (A + D).\sim B$$

TRUTH TABLE

A	B	D	~B	A+D	F
0	0	0	1	0	0
0	0	1	1	1	1
0	1	0	0	0	0
0	1	1	0	1	0
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	0	1	0
1	1	1	0	1	0

Table 1: Truth Table of the function $F = (A + D).\sim B$

BLOCK DIAGRAM

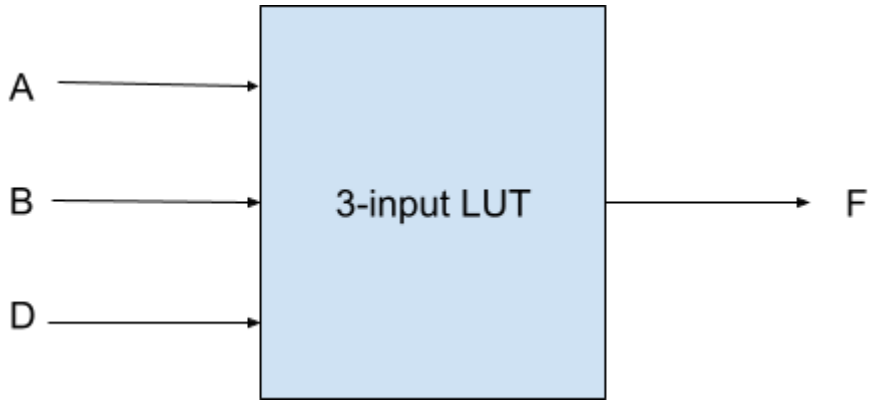

Figure 1: Block diagram of the LUT

SCHEMATIC

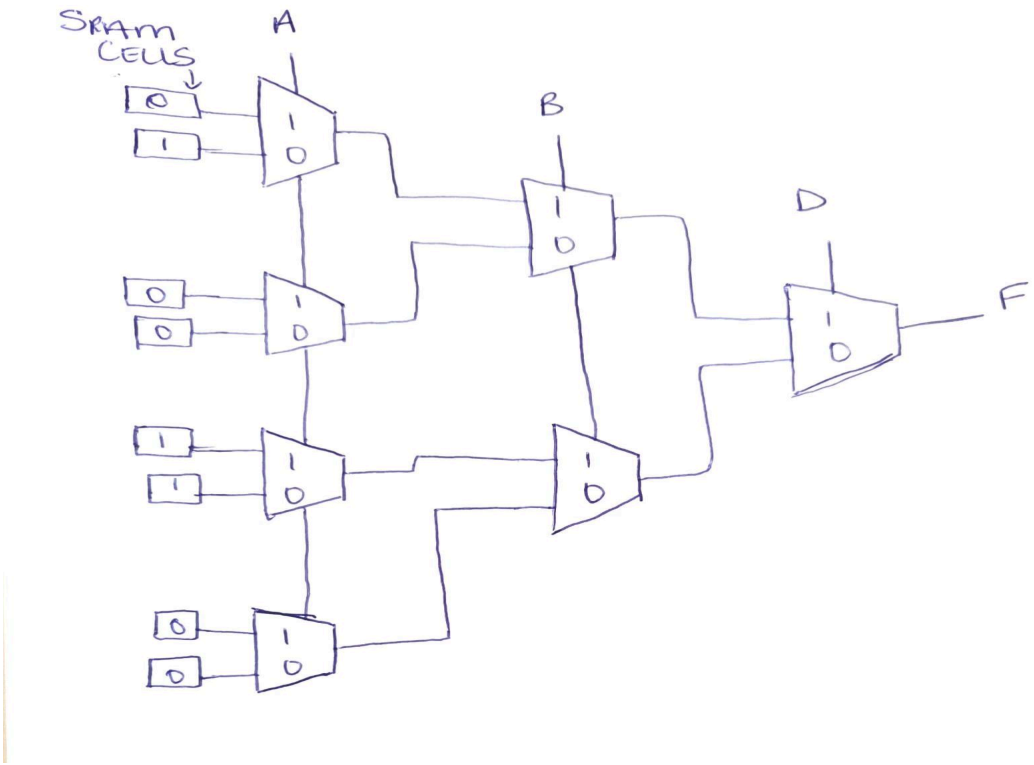

Figure 2: Schematic diagram of the 3-input LUT made using 2-to-1 MUX

CIRCUITRY DESIGN

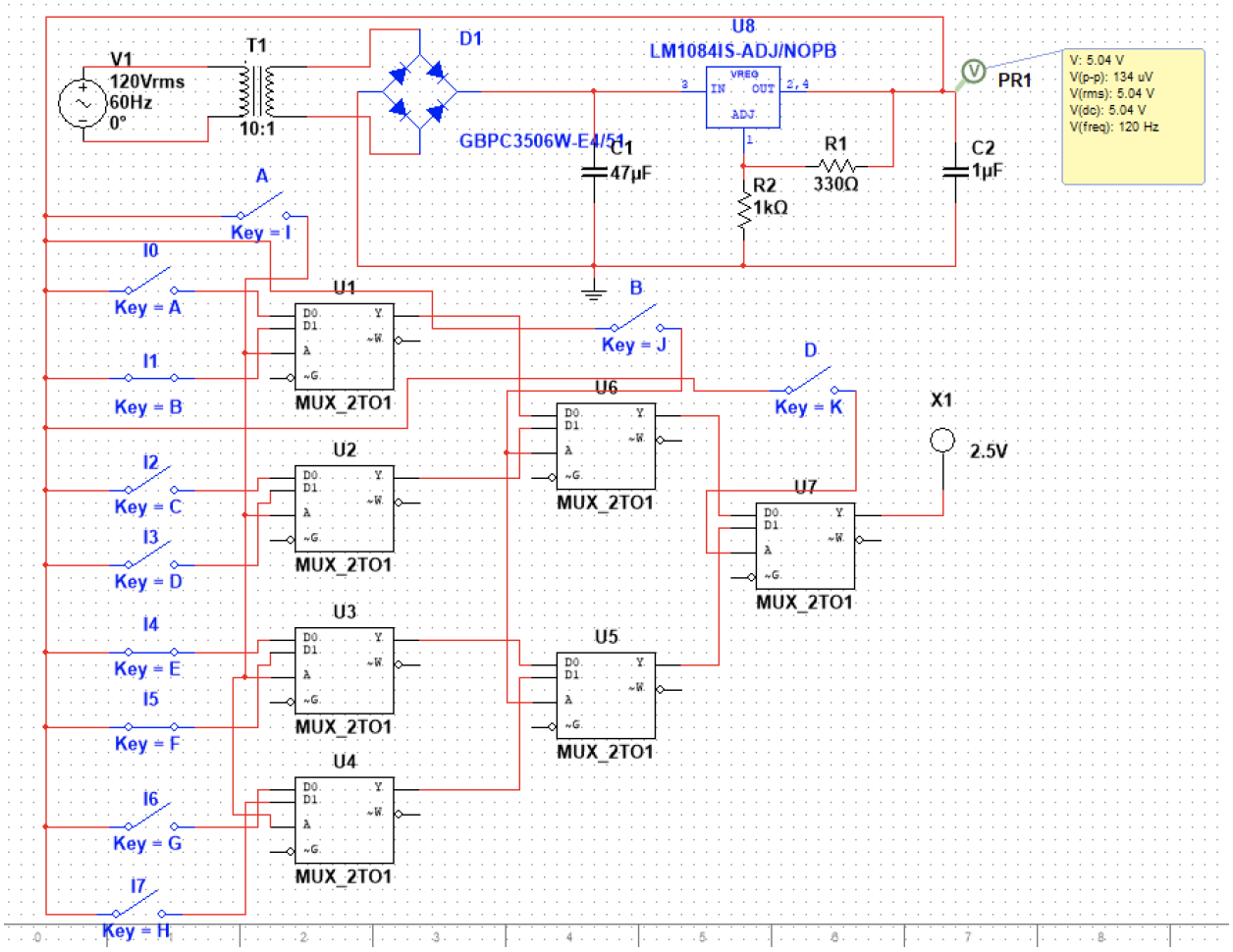

Figure 3: Simulated Circuit designed in Multisim

METHODOLOGY

The Boolean function $F = (A + D).\sim B$ was first analyzed and converted into a truth table to determine the required output for all input combinations. Based on the lab requirement to implement the design using Configurable Logic Block (CLB) components, the function was implemented using a 3-input LUT based on 2:1 multiplexers.

The physical implementation was carried out using three 74LS157N quad 2:1 multiplexer ICs. These multiplexers were configured to form the required LUT structure by appropriately wiring the select lines and data inputs according to the derived truth table. The active-low enable pins of the ICs were connected to ground to ensure continuous operation. The LUT configuration bits were implemented by connecting the multiplexer data inputs to either logic high or low according to the output values from the derived truth table.

A regulated 5V DC supply was used to power the circuit. This supply was generated from a previously constructed power supply circuit consisting of a transformer, full-bridge rectifier, filter and a voltage regulator.

The output of the final multiplexer stage was connected to an LED indicator through a 1kΩ current-limiting resistor. The LED was then connected to ground to provide visual confirmation of the output state.

For physical testing, the circuit was assembled on the breadboard and input combinations were applied manually connecting input wires to either 5V or ground. The output was verified by observing the LED state for each input configuration.

Components Used:

- BreadBoard
- 6V AC supply
- 2, 330Ω resistor
- 2, 1 kΩ resistor
- 4 1N007 diodes
- LM317 Regulator
- 3 SN74LS157N Quad 2 to 1 MUXES
- Wires
- Red LED
- Multimeter

Wiring Diagram

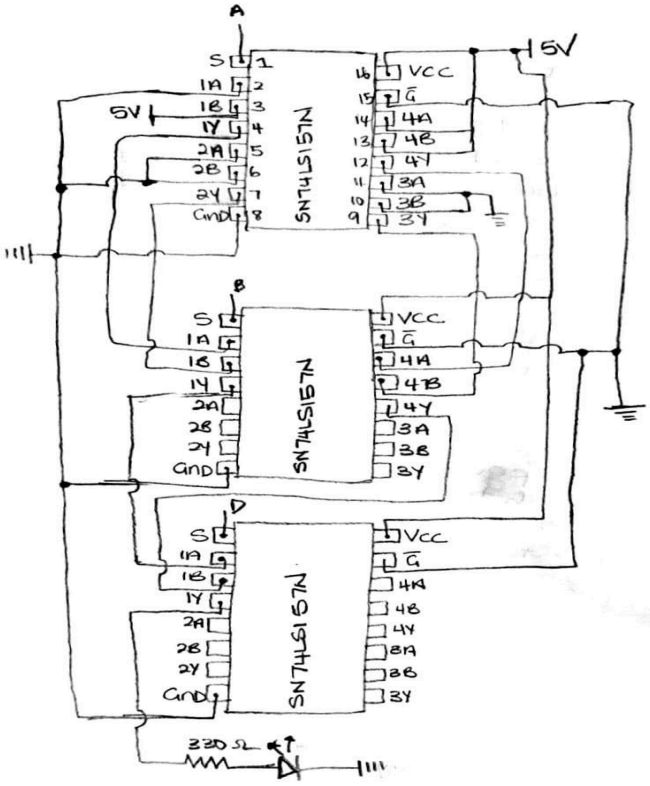

Figure 4: Wiring diagram of the physical implementation of the 3-input LUT

RESULTS

A	B	D	Theoretical F	Simulated F	Tested F
0	0	0	0	0	0
0	0	1	1	1	1
0	1	0	0	0	0
0	1	1	0	0	0
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	0	0	-
1	1	1	0	0	-

Table 2: Table showing the expected, simulated and tested results of the function $F = (A + D).\sim B$

Simulated Results

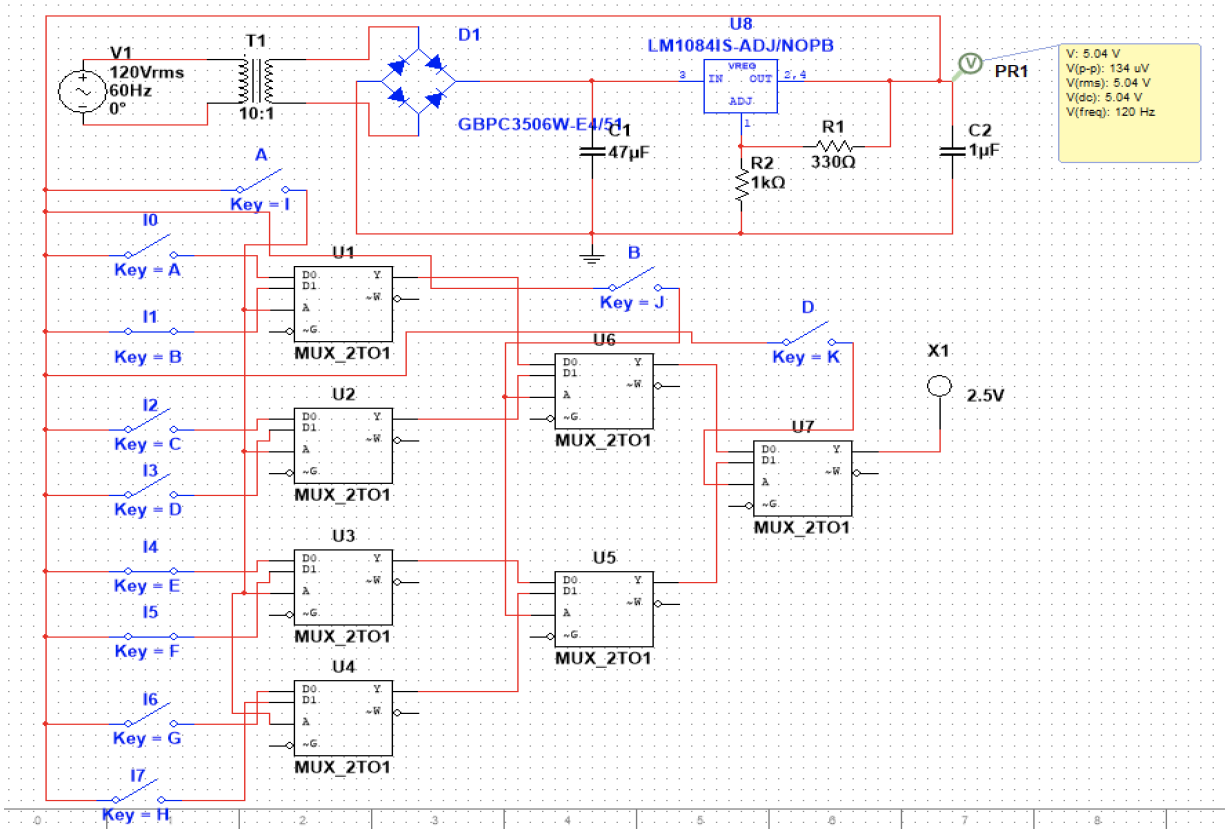

Figure 5: Simulated result for the input combination 000

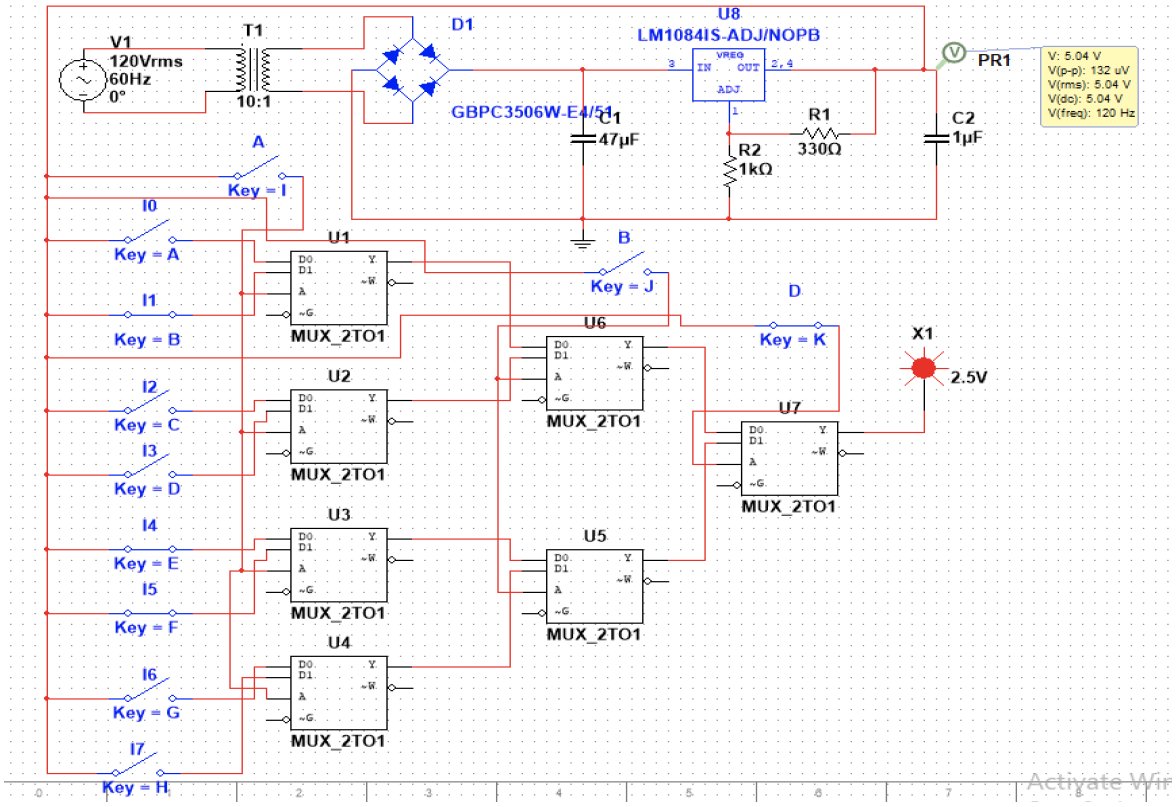

Figure 6: Simulated result for the input combination 001

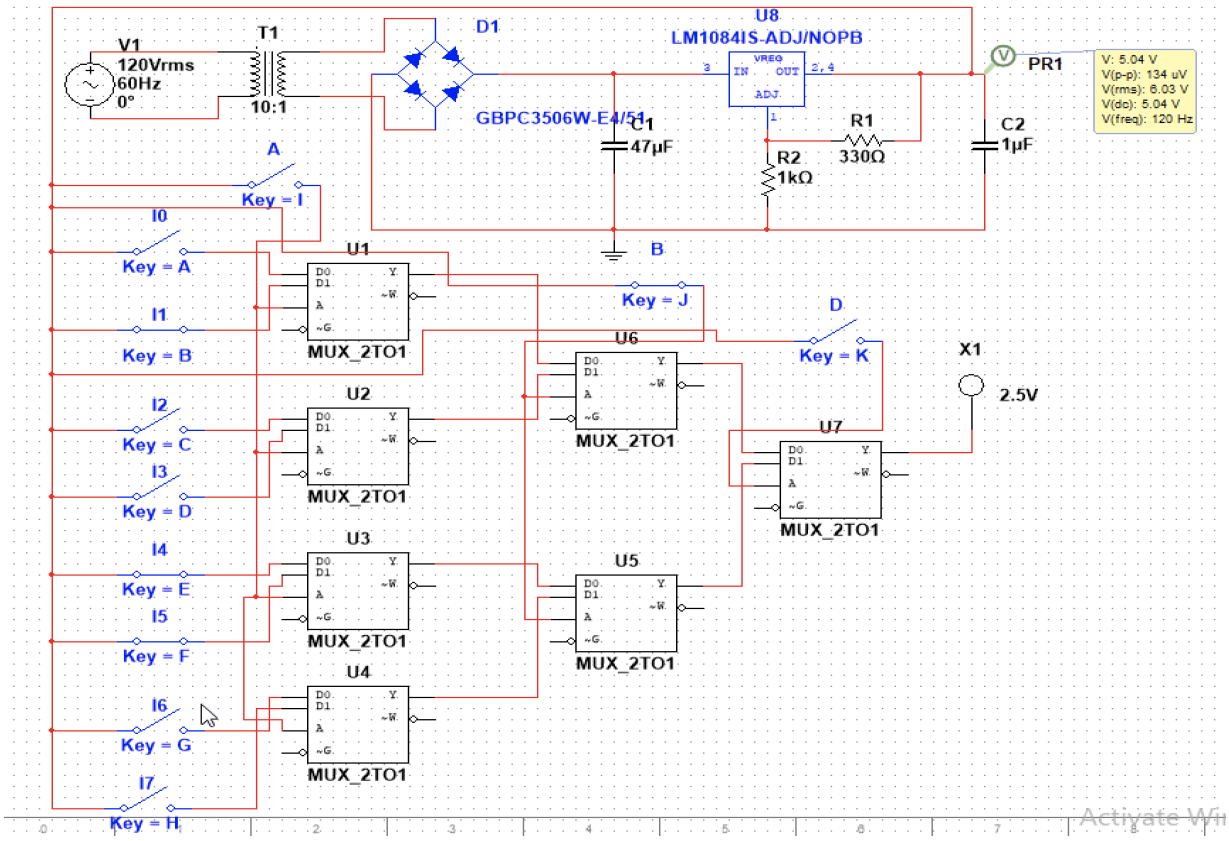

Figure 7: Simulated result for the input combination 010

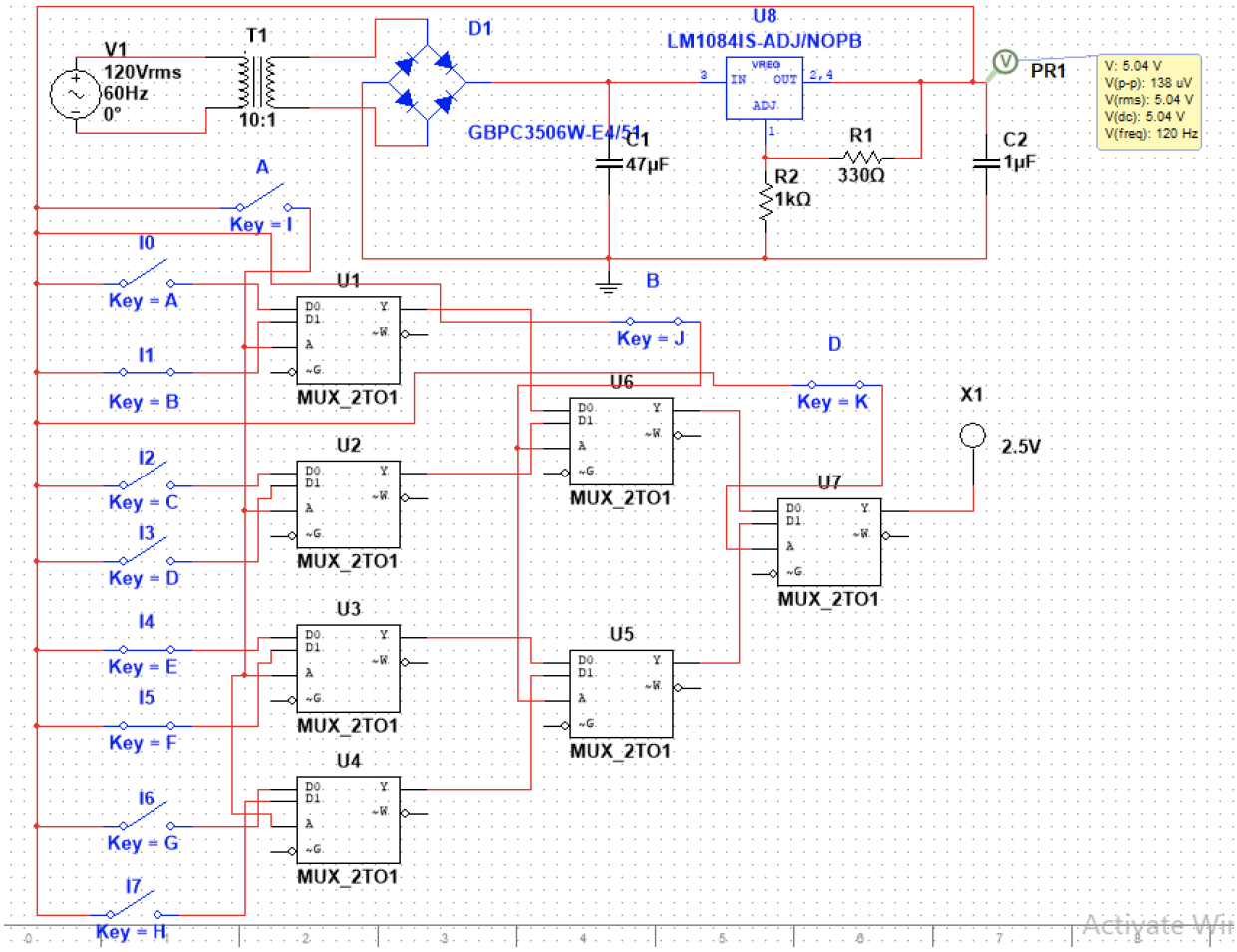

Figure 8: Simulated result for the input combination 011

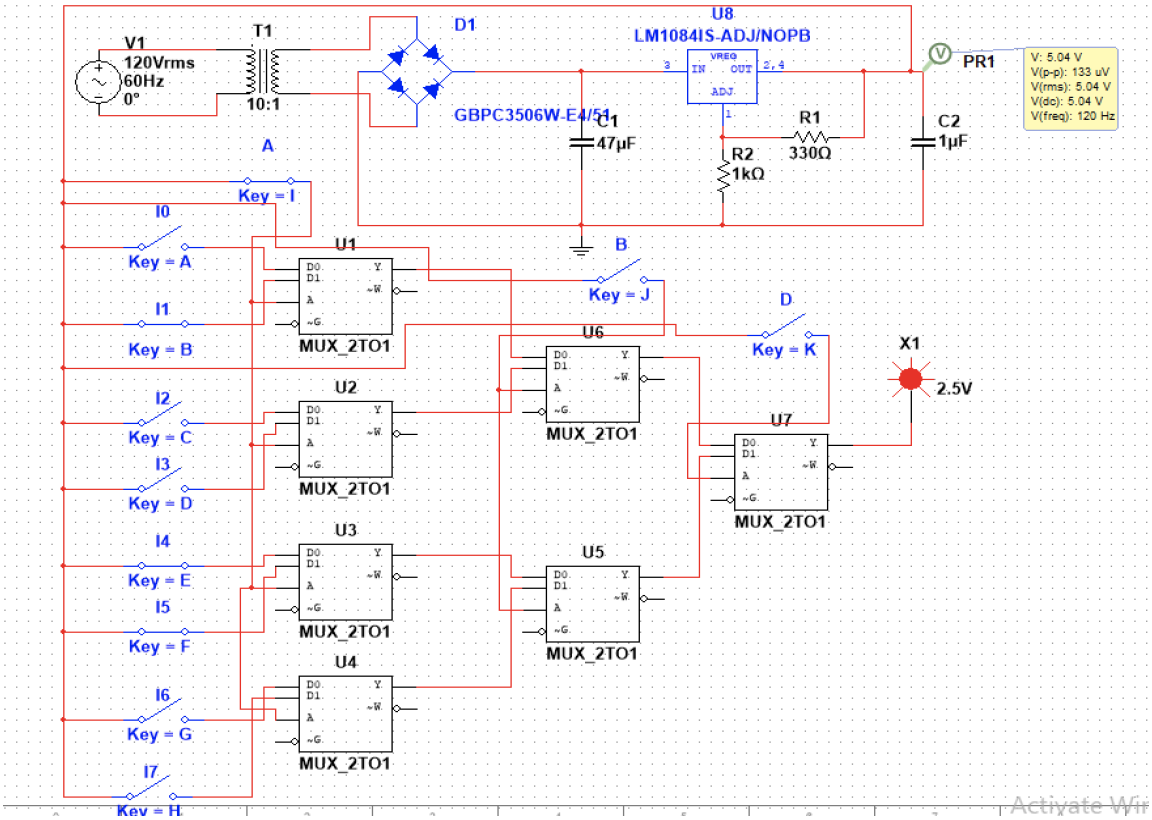

Figure 9: Simulated result for the input combination 100

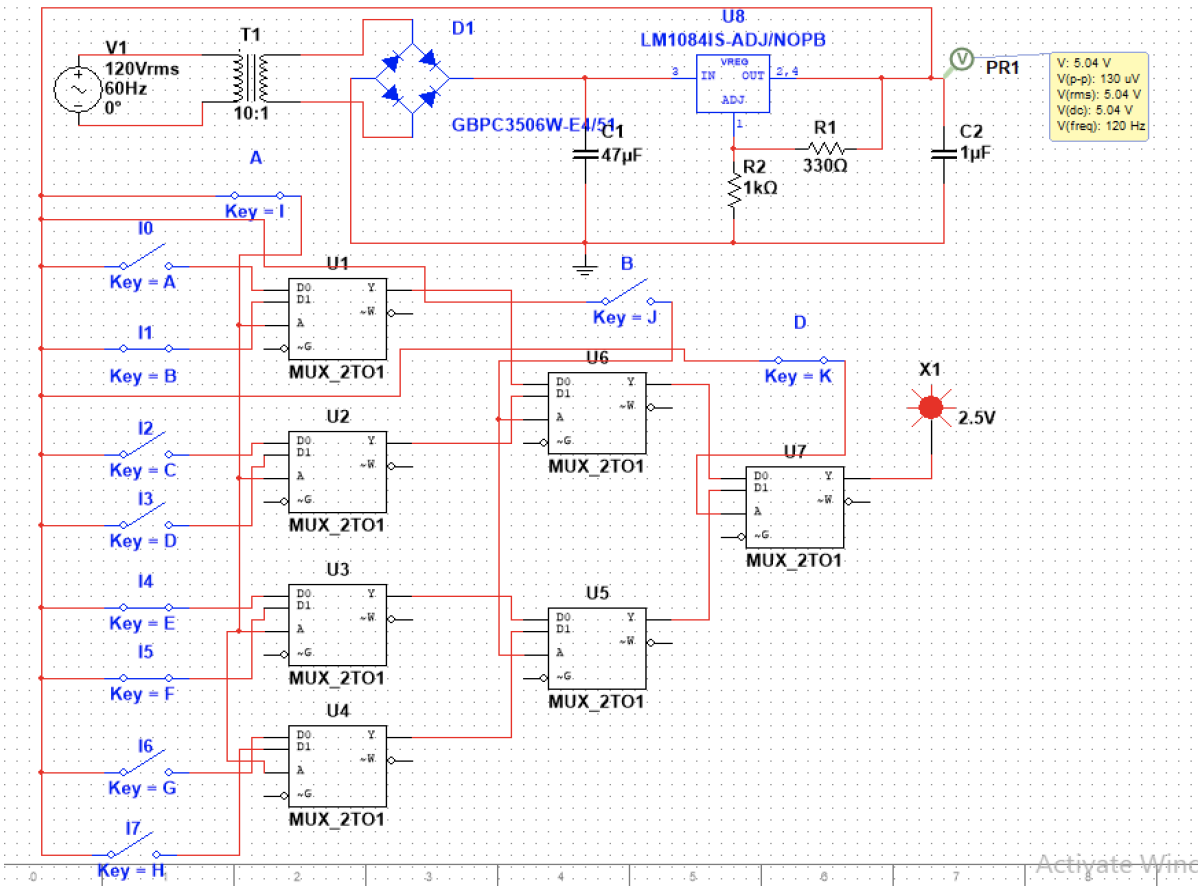

Figure 10: Simulated result for the input combination 101

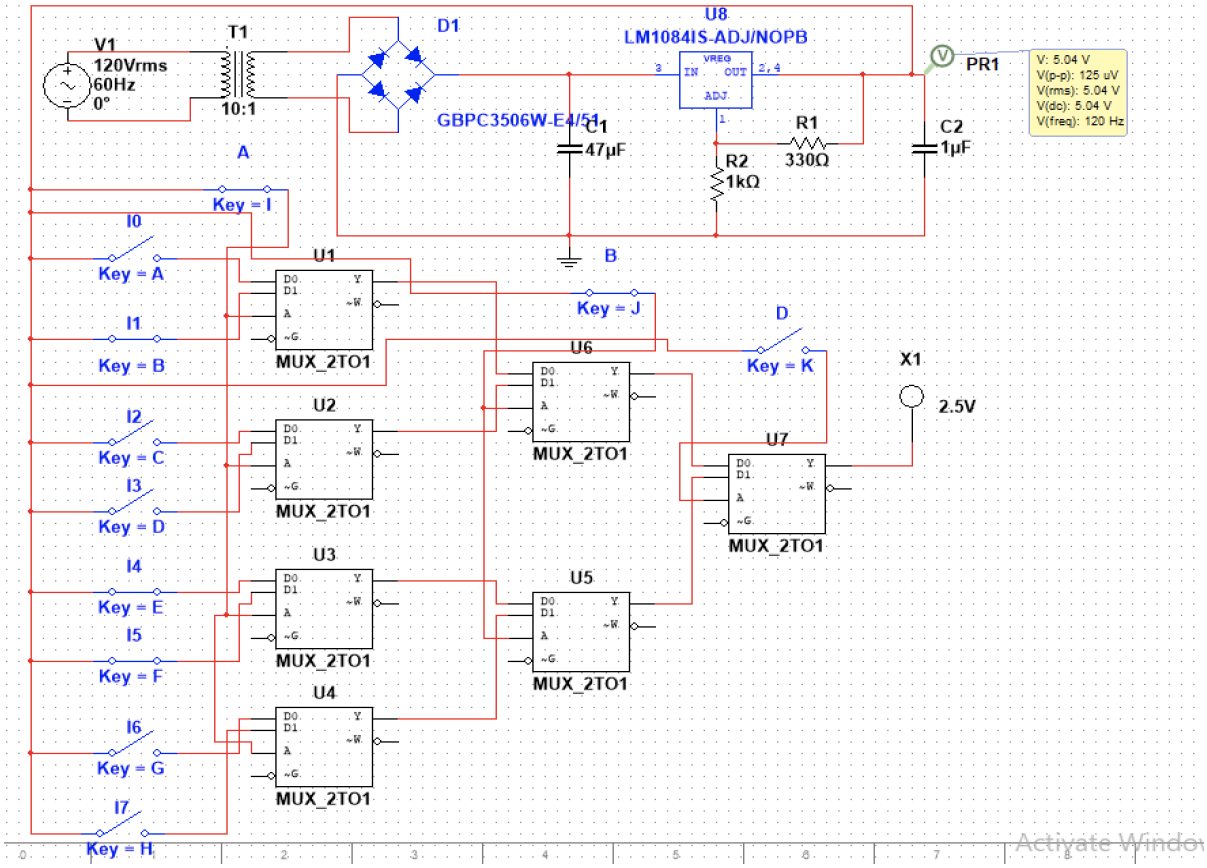

Figure 11: Simulated result for the input combination 110

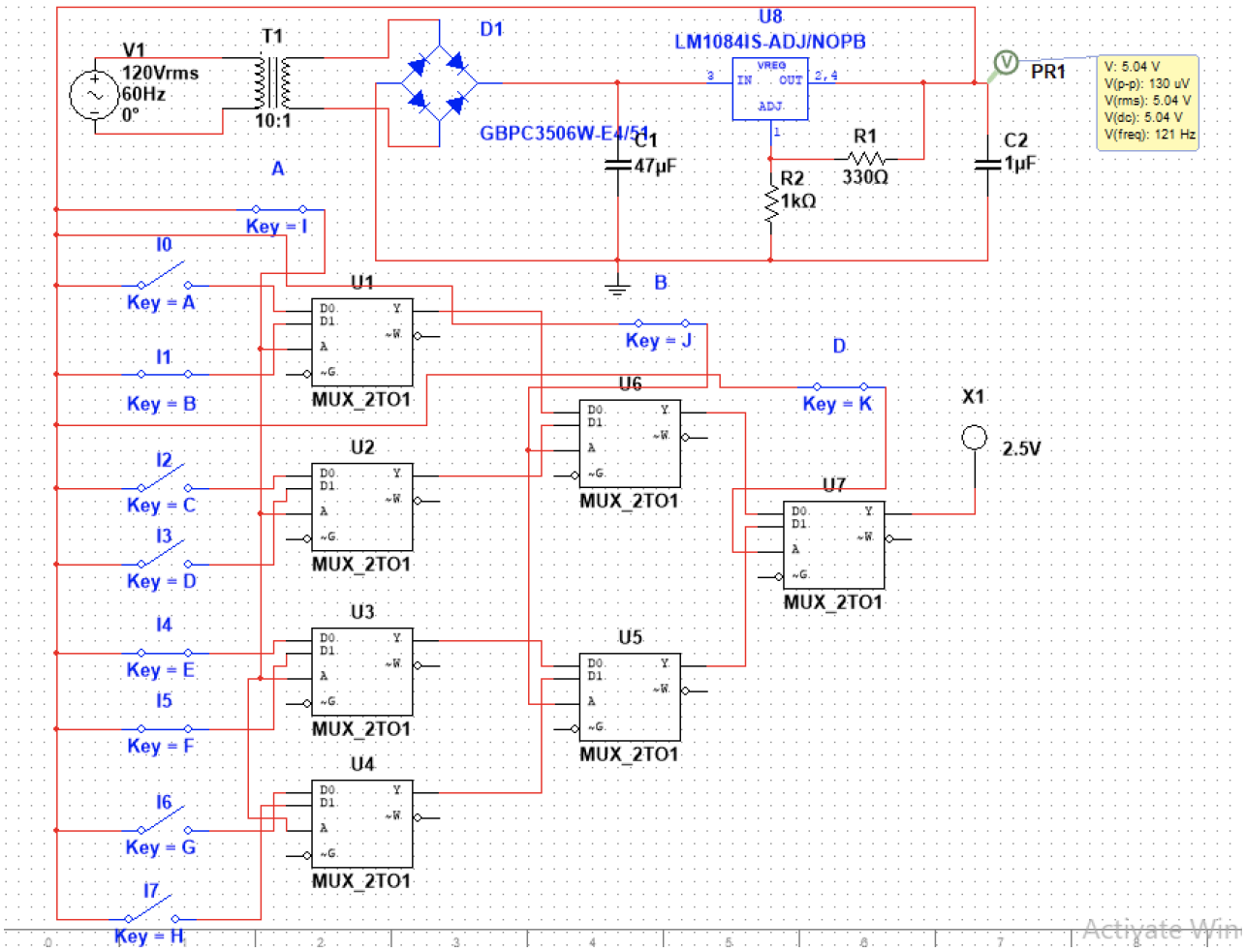

Figure 12: Simulated result for the input combination 111

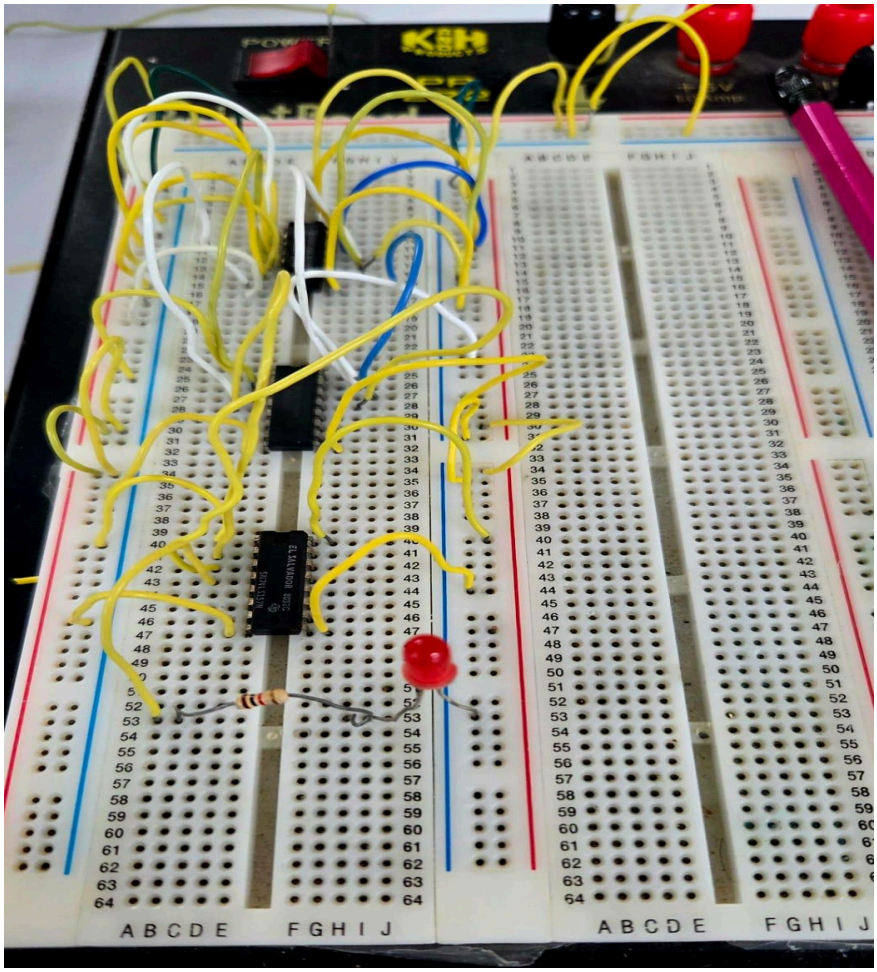

Figure 13: Physical result for the input combination 000

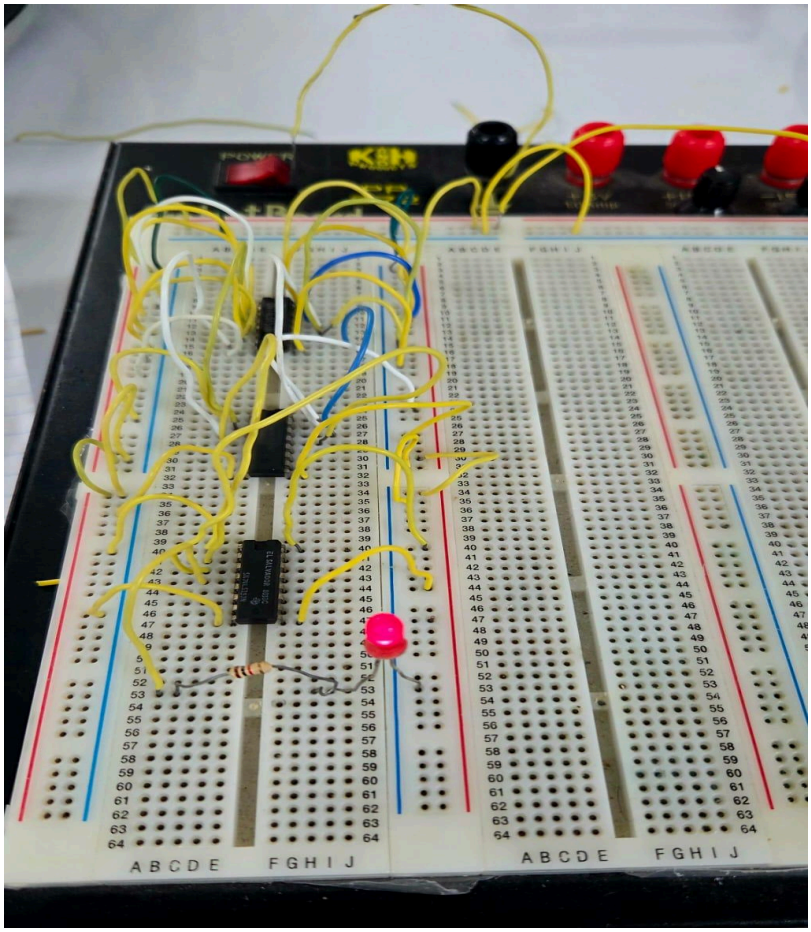

Figure 14: Physical result for the input combination 001

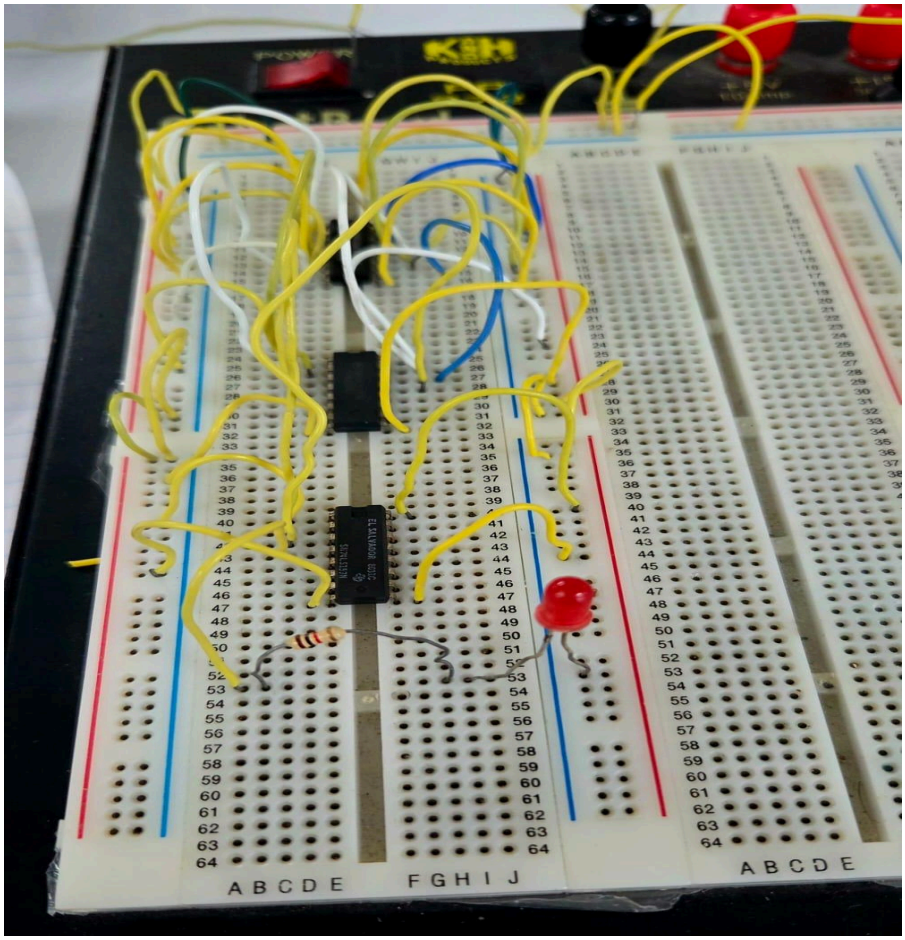

Figure 15: Physical result for the input combination 010

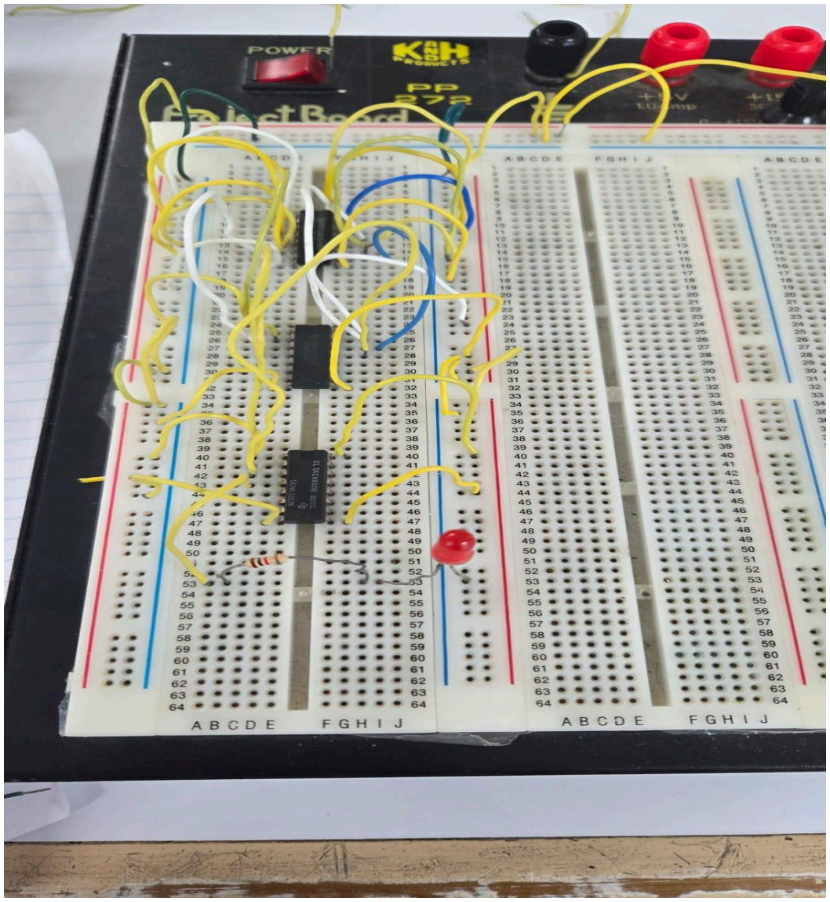

Figure 16: Physical result for the input combination 011

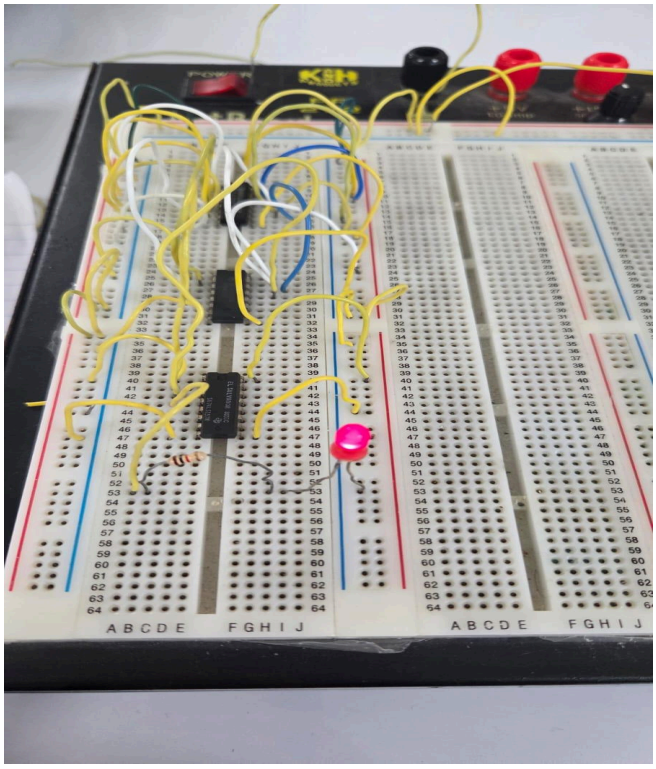

Figure 17: Physical result for the input combination 100

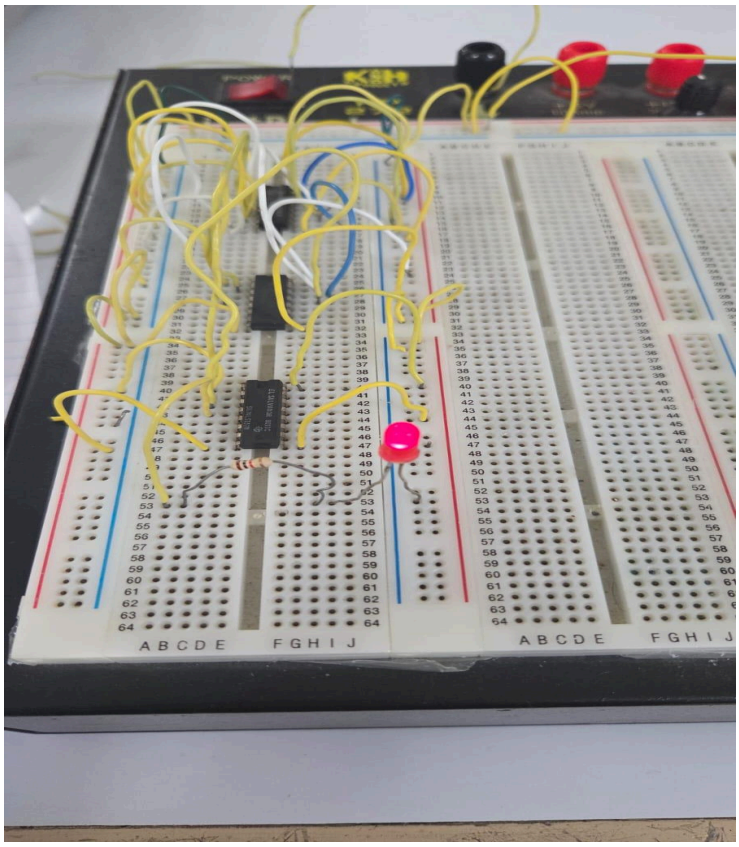

Figure 18: Physical result for the input combination 101

DISCUSSION

Table 1 shows the truth table of the function $F = (A + D).\sim B$, along with the expected output values. From figures 5-12, it is observed that the simulated results are consistent with the theoretical truth table for all tested input combinations, confirming correct implementation of the logic function in the simulation environment.

Additionally, figures 13-18 indicate that the physical testing also aligns with the expected function behavior for the majority of input states.

However, as shown in Table 2, input combinations 110 and 111 were not recorded during physical testing. This issue may have resulted from a loose connection, or wiring fault. The LED indicator was tested separately and confirmed to be functional, suggesting that the discrepancy was likely related to signal routing rather than component failure.

Overall, despite minor hardware inconsistencies during testing, the simulation results and majority of the physical tests confirm that the designed CLB correctly implements the assigned logic function.

CONCLUSION

This lab successfully demonstrated the design and implementation of a Configurable Logic Block (CLB) capable of implementing the Boolean function $F = (A + D).\sim B$. The function was first verified through theoretical analysis and truth table derivation, then implemented using a 3-input LUT structure based on 2:1 multiplexers. Simulation results obtained in Multisim matched the theoretical outputs for all input combinations, confirming correct logical implementation. The physical hardware implementation also produced results consistent with the simulated and theoretical results for the majority of tests. Minor discrepancies observed during testing were attributed to wiring and connectivity rather than component failure. Overall, the experiment successfully demonstrated LUT-based logic implementation using multiplexers.